Face Detection

1. Experiment Objective

Use MediaPipe Face Mesh to detect facial landmarks in the camera image, generating 468 facial mesh points and drawing contour connections.

2. Experiment Principle

1. Key Terms

Keyword	Description
MediaPipe Face Mesh	Google's facial mesh detection model, which outputs 468 3D facial landmarks covering the contours of facial features and the surface of the face
Face Mesh	A 468-point topology connected by triangular faces (Tessellation), finer than simple keypoints
Facial contours	Contour lines drawn from a specific subset of the 468 point indices (eyebrows, eyes, lips, outer face contour)

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start face detection
```

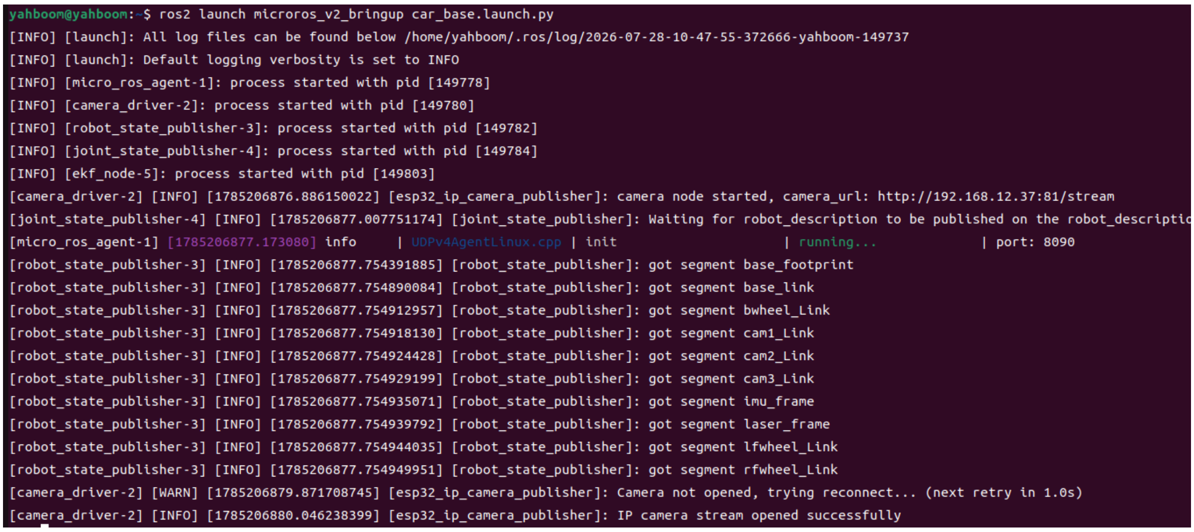
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑ Supported
No-PTZ version	☑ Supported
Required peripherals	ESP32 camera

2. Start the agent + camera + joint models (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_descriptio
[micro_ros_agent-1] [1785206877.173080] info     | UDPv4AgentLinux.cpp | init                     | running...             | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start face detection (Terminal 2)

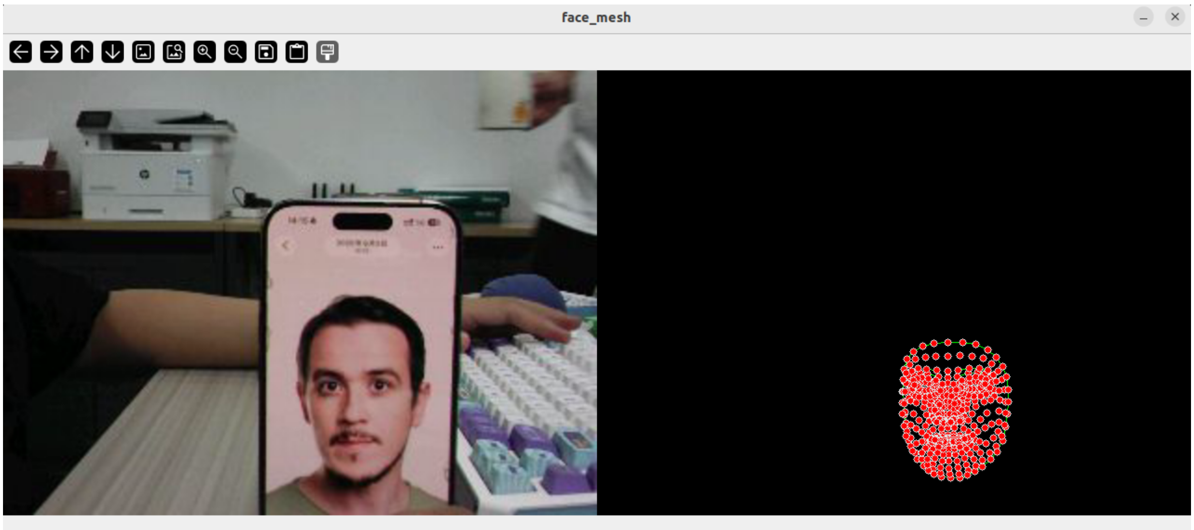
```
ros2 launch microros_v2_mediapipe_pkg face_mesh.launch.py
```

4. Operating Instructions

Press `q` to exit.

4. Observed Results

- The face is covered by 468 white dots, with triangular faces forming a fine facial surface mesh
- Facial contour lines (eyebrows, eyes, lips, outer face contour) are annotated in distinctive colors



5. Code Walkthrough

> Source code paths:
>
> - Launch: `~/microros_ws/src/microros_v2_mediapipe_pkg/launch/face_mesh.launch.py`
> - Node: `~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FaceMesh.py`

1. Core Node Analysis

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FaceMesh.p
y
class FaceMesh(Node):
    def __init__(self):
        super().__init__("face_mesh_node")
        self.mp_face_mesh = mp.solutions.face_mesh
        self.face_mesh = self.mp_face_mesh.FaceMesh(
            static_image_mode=False,
            max_num_faces=1,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        results = self.face_mesh.process(cv.cvtColor(frame, cv.COLOR_BGR2RGB))
        if results.multi_face_landmarks:
            for face_landmarks in results.multi_face_landmarks:
                self.mp_draw.draw_landmarks(
                    frame, face_landmarks,
                    self.mp_face_mesh.FACEMESH_TESSELATION, ...)
        cv.imshow(self.window_name, frame)
```

Key logic:

- `mp.solutions.face_mesh.FaceMesh()`: 468-point facial mesh detector
- `max_num_faces=1`: detects 1 face by default; increase it to support multiple faces
- `FACEMESH_TESSELATION`: triangular face topology that represents the facial surface more finely than skeleton connections
- Supports `FACEMESH_CONTOURS` (contour mode), which only draws facial contour lines (eyebrows, eyes, lips, outer contour) to reduce visual complexity

2. Key Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_mediapipe_pkg/launch/face_mesh.launch.py`
> - Node: `~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FaceMesh.py`

Parameter	Type	Default	Description
`max_num_faces`	int	1	Maximum number of faces to detect
`static_image_mode`	bool	false	Whether to detect each frame independently
`min_detection_confidence`	float	0.5	Detection confidence threshold
`min_tracking_confidence`	float	0.5	Tracking confidence threshold

3. Node Description

Node	Function
`face_mesh_node`	MediaPipe Face Mesh 468-point facial mesh detection + visualization

6. Frequently Asked Questions

Issue	Cause	Solution
Face not detected	Profile angle is too large	Face the camera directly
Frame lag / stutter	468-point mesh rendering is expensive	Switch to contour mode to reduce rendering complexity